Mini-Project: Reconstructing a Smooth Density Field and Its Hessian from Labelled Point Data with `GraphGP`

A First Feasibility Example

Extracted from "GP Field-Level Inference" collaboration proposal

Abel, Frank, Dodge, Clark, Wechsler, and collaborators

KIPAC / Stanford University

March 2026

Abstract

We describe a self-contained pilot project to test the core computational pipeline for Gaussian process (GP) density field reconstruction. The input is simple: N points in a periodic 3D box, each carrying two real-valued labels. The output is a smooth, continuous density field evaluated at the point positions, together with its gradient and Hessian—all with uncertainty estimates. We use `GraphGP`, a JAX-based library for scalable Gaussian processes, which achieves linear time and memory scaling in N. This document describes the problem setup, the mathematical framework, the computational pipeline step by step, and the deliverables. It is intended to be readable without specialized cosmological background.

1 The Problem

We are given:

- N points with positions $\boldsymbol{x}_i \in [0, L)^3$ in a periodic cubic box of side length L.
- Each point carries two real-valued labels, a_i and b_i (think of these as measured properties of each object—e.g., mass and temperature, or any two continuous quantities).

We want to answer three questions:

1. **What is the smooth density field?** The points are a noisy, discrete sampling of some underlying continuous density. We want to reconstruct that continuous field at each point position.
2. **What is the local geometry?** At each point, we want the gradient and Hessian (matrix of second derivatives) of the density field. These tell us whether the point sits at a peak, a valley, a ridge, or a saddle of the density.

3. **Do the labels depend on the local environment?** Once we have the density and its derivatives at each point, we can ask: do the labels a and b correlate with the density, with the curvature, or with the anisotropy of the local environment?

2 The Key Idea: Points Sample a Gaussian Random Field

The mathematical model is:

Step 1: There is a smooth underlying field. We assume there exists a continuous density contrast field $\delta(\boldsymbol{x})$ defined everywhere in the box. The word "density contrast" just means the fractional deviation from the average density: $\delta = (\rho - \bar{\rho})/\bar{\rho}$, so $\delta = 0$ is average, $\delta > 0$ is overdense, and $\delta < 0$ is underdense.

Step 2: The field is drawn from a Gaussian process. A Gaussian process (GP) is a probability distribution over functions. It is fully specified by two things:

- A *mean function* (we take this to be zero: no preferred density anywhere).
- A *covariance function* (or kernel) $C(r)$ that specifies how correlated the field is at two points separated by distance r.

Formally:

$$\delta \sim \mathcal{GP}\big(0,\ C(r)\big)\,. \tag{1}$$

The kernel $C(r)$ is a smooth, decaying function: points close together have correlated densities; points far apart are nearly independent. The *shape* of $C(r)$ is what we want to learn from the data.

Step 3: Points are drawn from the field. The observed points are a random sampling of the density field. Where the density is high, there are more points; where it is low, fewer. This is called an *inhomogeneous Poisson process*: the expected number of points in a small volume dV around position $\boldsymbol{x}$ is

$$\langle dN \rangle = \bar{n}\,(1 + \delta(\boldsymbol{x}))\,dV\,, \tag{2}$$

where $\bar{n} = N/L^3$ is the mean number density.

The inference problem. Given the observed points $\{\boldsymbol{x}_i\}$, we want to infer the smooth field $\delta(\boldsymbol{x})$ that generated them, learn the kernel $C(r)$, and compute derivatives of δ at each point position.

3 Why Derivatives Come for Free

A fundamental property of Gaussian processes is that **derivatives of a GP are themselves GPs**. If $\delta(\boldsymbol{x})$ is a GP with kernel $C(\boldsymbol{x}, \boldsymbol{x}')$, then the field value, gradient, and Hessian at

any point are jointly Gaussian:

$$\begin{pmatrix} \delta(\boldsymbol{x}) \\ \left.\dfrac{\partial \delta}{\partial x_i}\right|_{\boldsymbol{x}} \\ \left.\dfrac{\partial^2 \delta}{\partial x_i\, \partial x_j}\right|_{\boldsymbol{x}} \end{pmatrix} \quad \text{are jointly Gaussian,} \tag{3}$$

and their covariances are obtained by differentiating C. No extra model or separate computation is needed—the derivative information is already encoded in the kernel. `GraphGP` supports derivatives through custom JAX primitives [Dodge, Frank & Clark(2024)], so this is a built-in capability.

What the Hessian tells you. The Hessian $H_{ij} = \partial^2\delta/\partial x_i\, \partial x_j$ is a 3×3 symmetric matrix at each point. Its three eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3$ characterize the local geometry:

Local geometry	λ_1	λ_2	λ_3
Peak (local maximum)	$-$	$-$	$-$
Ridge (filament-like)	$-$	$-$	$+$
Saddle (sheet-like)	$-$	$+$	$+$
Valley (void-like)	$+$	$+$	$+$

(The sign convention here is that of $-H_{ij}$, so that peaks have all-negative eigenvalues of the Hessian itself.) The eigenvectors give the principal directions of curvature—the orientation of the ridge or saddle.

Uncertainty on derivatives. Because the derivatives are part of the joint GP posterior, each eigenvalue comes with an uncertainty estimate, propagated from the data. A point near a filament/sheet boundary will have large uncertainty in its classification.

4 The Computational Pipeline

The pipeline has five steps. We describe each concretely.

4.1 Step 1: Set Up the Data

- Load N point positions $\boldsymbol{x}_i \in [0, L)^3$ and labels (a_i, b_i).
- The box is periodic: points near the boundary are neighbours of points near the opposite face. `GraphGP` handles periodic boundaries natively.

In code (schematic):

```
import jax.numpy as jnp
points = jnp.load("points.npy")      # shape (N, 3)
labels_a = jnp.load("labels_a.npy")  # shape (N,)
labels_b = jnp.load("labels_b.npy")  # shape (N,)
```

4.2 Step 2: Build the Neighbour Graph

`GraphGP` uses the *Vecchia approximation*: instead of building the full $N \times N$ covariance matrix (impossible for large N), it approximates the GP by conditioning each point on only its k nearest previously generated neighbours. This requires a neighbour graph.

```
import graphgp as gp

graph = gp.build_graph(points, n0=100, k=10)
```

Here `n0` is the number of points treated exactly at the coarsest level, and `k` is the number of conditioning neighbours per point. Typical choices are `n0` $\sim$ 50–200 and `k` $\sim$ 5–30; larger k is more accurate but costs more. The graph construction is $\mathcal{O}(N \log N)$.

4.3 Step 3: Choose and Parameterize the Kernel

The kernel $C(r)$ encodes our assumption about the smoothness and correlation length of the density field. For this pilot, a good starting point is the radial basis function (RBF, or squared-exponential) kernel:

$$C(r) = \sigma^2 \exp\left(-\frac{r^2}{2\ell^2}\right), \tag{4}$$

where σ^2 is the variance (amplitude of fluctuations) and ℓ is the correlation length (how far away points are still correlated).

`GraphGP` takes a discretized kernel: a table of $(r_i, C(r_i))$ values that it linearly interpolates. This makes it easy to use any kernel shape, including empirical or non-parametric kernels.

```
covariance = gp.extras.rbf_kernel(
    variance=1.0,  # will be optimized
    scale=0.3,     # will be optimized
    r_min=1e-4, r_max=L/2,
    n_bins=1000, jitter=1e-4
)
```

The parameters σ^2 and ℓ (and potentially the full kernel shape) will be learned from the data in Step 5.

4.4 Step 4: Generate and Invert — The Core GP Operations

Given the graph and kernel, `GraphGP` provides three operations that form the computational backbone:

(a) Generate: Apply the approximate Cholesky factor L to a white-noise vector $\boldsymbol{\xi}$ to produce a correlated GP realization: $\boldsymbol{\delta} = L\,\boldsymbol{\xi}$. Cost: $\mathcal{O}(Nk^3)$.

(b) Solve: Apply the inverse Cholesky factor $L^{-1}\boldsymbol{\delta}$. This "whitens" a field realization, undoing the correlations. Cost: $\mathcal{O}(Nk^3)$.

(c) Log-determinant: Compute $\ln\det L$, needed for the GP log-probability. Cost: $\mathcal{O}(Nk^3)$.

All three are differentiable through JAX, meaning we can compute gradients of any function that uses them—including the log-posterior and its derivatives with respect to kernel parameters.

4.5 Step 5: Infer the Field and Learn the Kernel

The inference proceeds by maximizing (or sampling) the posterior probability of the field and kernel parameters given the observed points.

The log-posterior. Combining the Poisson likelihood (Eq. 2) and the GP prior (Eq. 1):

$$\ln p(\boldsymbol{\delta}, \theta \,|\, \text{data}) = \underbrace{\sum_{i=1}^{N} \ln\big[\bar{n}(1+\delta_i)\big] - \bar{n}\int (1+\delta)\, dV}_{\text{Poisson log-likelihood}} \underbrace{-\frac{1}{2}\,\boldsymbol{\delta}^\top C_\theta^{-1}\boldsymbol{\delta} - \frac{1}{2}\ln\det C_\theta}_{\text{GP log-prior}} + \text{const.} \quad (5)$$

Here θ denotes the kernel parameters (e.g., σ^2 and ℓ for the RBF kernel, or a set of band-powers for a non-parametric kernel), and $\delta_i = \delta(\boldsymbol{x}_i)$ is the field at the i-th point.

The integral in the Poisson term is over the full box volume. For a periodic box it can be approximated on a coarse auxiliary grid or evaluated analytically if the field is represented on a grid for this purpose alone.

Optimization. For a first pass, find the maximum a posteriori (MAP) field and kernel parameters by gradient ascent on Eq. (5). JAX computes the gradient automatically:

```
import jax

def log_posterior(delta, theta, points, graph):
    # Poisson log-likelihood
    n_bar = len(points) / L**3
    ll = jnp.sum(jnp.log(n_bar * (1 + delta)))
    ll -= n_bar * L**3  # integral term (periodic box)

    # GP log-prior via GraphGP
    C = make_kernel(theta)
    white = gp.solve(graph, C, delta)  # L^{-1} delta
    logdet = gp.logdet(graph, C)
    lp = -0.5 * jnp.dot(delta, white) - 0.5 * logdet

    return ll + lp

grad_fn = jax.grad(log_posterior, argnums=(0, 1))
```

This gradient can be fed to any optimizer (L-BFGS, Adam) or to a Hamiltonian Monte Carlo (HMC) sampler for full posterior exploration.

Sampling (optional, for uncertainty). For full uncertainty quantification, sample from the posterior using HMC. Libraries such as `blackjax` or NIFTy's variational inference integrate directly with JAX:

```
import blackjax
kernel = blackjax.nuts(log_posterior, ...)
states = blackjax.run(kernel, num_samples=1000)
```

Each posterior sample is a full realization of the field δ_i at all N points, together with the kernel parameters θ.

4.6 Step 6: Compute Derivatives

Once we have the posterior field (MAP or samples), the gradient and Hessian at each point follow from the GP derivative formulas (Section 3). `GraphGP` provides these via its derivative-aware JAX primitives. Schematically:

```
# Gradient: d(delta)/dx_i at each point
grad_delta = gp.gradient(graph, covariance, delta)
                                        # shape (N, 3)

# Hessian: d^2(delta)/dx_i dx_j at each point
hess_delta = gp.hessian(graph, covariance, delta)
                                        # shape (N, 3, 3)
```

From the Hessian, compute the eigenvalues and eigenvectors:

```
eigenvalues, eigenvectors = jnp.linalg.eigh(hess_delta)
                            # eigenvalues: shape (N, 3)
                            # eigenvectors: shape (N, 3, 3)
```

If posterior samples are available, repeat for each sample to obtain uncertainty on the eigenvalues.

5 Deliverables

The pipeline produces the following outputs, all defined at the N point positions:

5.1 Per-Point Quantities (Table)

Quantity	Shape	Source	What it tells you
δ_i	$(N,)$	GP posterior	Local density contrast
$(\partial_1\delta,\ \partial_2\delta,\ \partial_3\delta)_i$	$(N,3)$	GP derivatives	Direction of steepest density change
H_{ij}	$(N,3,3)$	GP derivatives	Curvature of the density field
$\lambda_1, \lambda_2, \lambda_3$	$(N,3)$	Eigendecomposition of H	Local geometry classification
$\hat{e}_1, \hat{e}_2, \hat{e}_3$	$(N,3,3)$	Eigenvectors of H	Principal directions of curvature
$\nabla^2\delta = \mathrm{tr}(H)$	$(N,)$	Trace of H	Convergence / divergence
$s^2 = s_{ij}\, s_{ij}$	$(N,)$	Traceless part of H	Tidal shear strength
a_i	$(N,)$	Input data	Label a
b_i	$(N,)$	Input data	Label b

If posterior samples are used, each row becomes a distribution rather than a point estimate: the mean, standard deviation, and full covariance between quantities are available.

5.2 Global Quantities

Quantity	Source	What it tells you
Kernel $C(r)$ or band-powers $P(k_i)$	Optimized hyperparameters	Correlation structure of the fiel
Fisher matrix F_{ij} for band-powers	Hessian of log-marginal likelihood	Uncertainty on $P(k_i)$ estimates

5.3 Downstream Analysis: Do Labels Depend on Environment?

With the per-point table in hand, the central scientific question becomes straightforward to investigate. For each point we now have the density δ_i, the tidal shear s_i^2, the Laplacian $\nabla^2\delta_i$, the eigenvalues $\lambda_{1,2,3}$, and the labels a_i, b_i. We can then ask a sequence of increasingly refined questions:

(Q1) Do labels correlate with density? Compute $\langle a \,|\, \delta \rangle$ and $\langle b \,|\, \delta \rangle$ by binning or GP regression of labels on δ. A non-trivial dependence means the labels are sensitive to the local density environment.

(Q2) Do labels depend on more than density? At fixed δ, do labels correlate with the tidal shear s^2 or the Laplacian $\nabla^2\delta$? Concretely: compute partial correlations $\mathrm{Corr}(a, s^2 \,|\, \delta)$. A non-zero result means the labels carry information about the anisotropy of the environment beyond what density alone provides.

(Q3) Do labels depend on the orientation of the tidal field? At fixed δ and s^2, do labels correlate with the eigenvalue ratios (prolateness λ_1/λ_3, oblateness λ_2/λ_3)? This probes whether the labels are sensitive to whether the local geometry is filament-like versus sheet-like.

(Q4) Do differently-labelled points cluster differently? Split points into subsamples by label (e.g., above/below median a). Compute the kernel $C(r)$ separately for each subsample and for the cross-correlation. The ratio of kernels directly measures label-dependent clustering.

Each of these questions can be addressed with propagated uncertainty from the GP posterior. If posterior samples are available, the uncertainty on every correlation and partial correlation is obtained by repeating the analysis on each sample.

6 Practical Considerations

Scaling. The entire pipeline scales as $\mathcal{O}(Nk^3)$ per evaluation of the log-posterior and its gradient. For $N = 10^5$ points and $k = 10$, a single evaluation takes on the order of seconds on a GPU. MAP optimization requires ~ 100 evaluations; HMC sampling requires ~ 1000. The full pipeline for 10^5 points should run in minutes to hours on a single GPU.

Choice of k. Larger k makes the Vecchia approximation more accurate but increases cost as k^3. For smooth, slowly decaying kernels, $k = 10$–20 is typically sufficient. One can check convergence by increasing k and verifying that the results do not change.

Periodic boundaries. `GraphGP` builds the neighbour graph using distances that wrap around the periodic box. No special handling is needed beyond specifying the box size.

Kernel choice. The RBF kernel is a reasonable default for smooth fields. For fields with more small-scale structure, a Matérn kernel (which allows tunable smoothness) may be more appropriate. The non-parametric approach—fitting band-powers $P(k_i)$ at discrete wavenumbers—is the most flexible and lets the data determine the kernel shape.

The Poisson integral term. The integral $\int (1+\delta)\, dV$ in the Poisson log-likelihood requires knowing the field everywhere, not just at the data points. For a periodic box, the simplest approach is to deposit the GP field onto a coarse regular grid (via interpolation from the GP posterior at grid points) and sum. Alternatively, the constraint $\int \delta\, dV = 0$ (the field has zero mean by construction) makes this term equal to the box volume L^3, simplifying the computation.

Jitter. Adding a small diagonal term $\epsilon \sim 10^{-4}$ to the kernel at $r = 0$ ("jitter") prevents numerical singularities when points are very close together. This is standard practice and is included in `GraphGP`'s kernel construction.

7 Suggested Milestones

Week 1–2: Generate a test dataset. Draw a GP realization on a grid using a known kernel, Poisson-sample points from the resulting density field, and assign labels that depend on local density and tidal shear (with known functional form). This provides ground truth for validation.

Week 3–4: Implement the log-posterior (Eq. 5) using `GraphGP`. Optimize for the field δ_i at fixed kernel parameters (known truth). Verify that the reconstructed field matches the true field.

Week 5–6: Promote kernel parameters to optimization targets. Verify that the learned $C(r)$ (or band-powers) match the known truth. Compute the Fisher matrix and verify that the uncertainties are consistent with the scatter across multiple realizations.

Week 7–8: Implement the derivative computation (gradient and Hessian). Verify eigenvalue classification against the known tidal field from the ground-truth GP realization.

Week 9–10: Run the label–environment correlation analysis (Q1–Q4 above). Verify that the known dependence of labels on density and tidal shear is recovered, and that false positives are absent for the independent label.

Week 11–12: Implement HMC sampling for full posterior uncertainty. Produce the complete per-point table with error bars. Write up results.

8 Software Dependencies

Package	Role	Source
`GraphGP`	Scalable GP (Vecchia)	https://github.com/stanford-ism/graphgp
JAX	Autodiff, GPU acceleration	https://github.com/google/jax
`blackjax`	HMC / NUTS sampling	https://github.com/blackjax-devs/blackjax
`optax`	Gradient-based optimization	https://github.com/google-deepmind/optax

References

References

[Dodge, Frank & Clark(2024)] Dodge, B., Frank, P., & Clark, S. E. 2024, `GraphGP`: Scalable Gaussian Processes using Nearest Neighbor Graphs. https://github.com/stanford-ism/graphgp

[Edenhofer et al.(2022)] Edenhofer, G., Leike, R. H., Frank, P., & Enßlin, T. A. 2022, arXiv:2206.10634. Sparse Kernel Gaussian Processes through Iterative Charted Refinement (ICR).

[Edenhofer et al.(2024)] Edenhofer, G., Zucker, C., Frank, P., et al. 2024, A&A, 685, A82. A parsec-scale Galactic 3D dust map out to 1.25 kpc from the Sun.

[Katzfuss & Guinness(2021)] Katzfuss, M., & Guinness, J. 2021, Statistical Science, 36, 124. A General Framework for Vecchia Approximations of Gaussian Processes.

[Banerjee & Abel(2021)] Banerjee, A., & Abel, T. 2021, MNRAS, 500, 5479. Nearest neighbour distributions: New statistical measures for cosmological clustering.